

软件概要设计说明书

属性	内容
系统名称	松果集市
编制单位	软件工程基础第 3 组
修订日期	2026-08-28

修订记录

版本	日期	修订说明
V1.0	2026 春	初版文档。
V2.1	2026-08-28	同步当前系统架构、服务端购物车与地址、容器与 Kubernetes 部署及自动化测试设计。
V2.2	2026-08-29	统一用例编号，按现有图片整理组件级动态模型覆盖，校正容器版本、订单现状和历史测试状态。

目录

软件概要设计说明书

修订记录

目录

1 项目概述

- 1.1 标识
- 1.2 系统概述
- 1.3 文档概述
- 1.4 参考资料
- 1.5 术语与缩略语

2 设计概述

- 2.1 设计目标
- 2.2 设计原则
- 2.3 运行环境
- 2.4 设计约束
- 2.5 假设与依赖

3 系统总体设计

- 3.1 总体架构
- 3.2 分层设计
- 3.3 架构选型说明
- 3.4 角色与权限模型
- 3.5 总体业务流程

4 子系统与模块设计

- 4.1 模块划分
- 4.2 用户与权限模块设计
 - 4.2.1 模块职责

- 4.2.2 主要子功能
 - 4.2.3 对外接口
- 4.3 商品与内容模块设计
 - 4.3.1 模块职责
 - 4.3.2 商品状态设计
 - 4.3.3 设计要点
- 4.4 搜索与推荐模块设计
 - 4.4.1 模块职责
 - 4.4.2 设计要点
- 4.5 购物车与订单模块设计
 - 4.5.1 模块职责
 - 4.5.2 当前实现
 - 4.5.3 目标订单状态设计
- 4.6 售后与评价模块设计
 - 4.6.1 当前能力
 - 4.6.2 待实现能力
- 4.7 平台治理模块设计
 - 4.7.1 当前能力
 - 4.7.2 后续能力
- 4.8 AI 扩展模块设计
 - 4.8.1 模块职责
 - 4.8.2 设计要点
- 4.9 基础支撑模块设计
 - 4.9.1 模块职责
 - 4.9.2 设计要点
- 4.10 业务用例组件级动态模型整理
 - 4.10.1 UC-01 注册并登录
 - 4.10.2 UC-06 卖家发布并维护商品
 - 4.10.3 UC-07 管理员审核商品
 - 4.10.4 UC-08 维护购物车和地址并创建订单
 - 4.10.5 UC-09 查询订单并提交评价
 - 4.10.6 UC-10 创建话题并完成社区互动
 - 4.10.7 UC-11 建立会话、收发消息并使用 AI 议价
 - 4.10.8 UC-12 提交实名认证并由管理员审核
 - 4.10.9 UC-13 使用 AI 购物助手
 - 4.10.10 当前未覆盖项
- 5 接口概要设计
 - 5.1 外部接口设计
 - 5.2 API 设计规范
 - 5.2.1 请求与响应规范
 - 5.2.2 通用状态码
 - 5.3 主要接口分组
 - 5.4 内部接口设计原则
- 6 数据概要设计
 - 6.1 数据架构概述
 - 6.2 核心实体设计
 - 6.3 数据关系说明
 - 6.4 数据库设计原则
 - 6.5 缓存设计
- 7 关键设计说明
 - 7.1 安全设计
 - 7.1.1 身份认证与权限控制
 - 7.1.2 输入与内容安全

7.1.3 数据与隐私保护
7.2 性能设计
7.3 可靠性设计
7.4 可维护性设计
7.5 可扩展性设计
8 部署概要设计
8.1 部署架构
8.2 部署方式
8.3 环境划分
8.4 部署与运维要求
9 设计验证与测试考虑
9.1 设计验证重点
9.2 测试要求
10 设计风险与未决事项
10.1 设计风险
10.2 未决事项
11 需求与设计对应关系

1 项目概述

1.1 标识

- 系统名称：松果集市（校园新品与二手交易平台）
- 系统标识号：TRADE-PLATFORM-2026
- 文档名称：软件概要设计说明书
- 文档状态：当前代码与部署基线
- 编制单位：软件工程基础第 3 组

1.2 系统概述

松果集市是一个面向高校师生的校园新品与二手交易平台，目标是为用户提供安全、便捷、可信赖的商品流通环境。系统围绕用户注册登录、实名认证、商品发布和审核、商品浏览检索、收藏关注、购物车、订单、评价、社区话题、站内消息和 AI 辅助等场景组织业务。

系统采用前后端分离架构。前端以 Vue 3 和 uni-app 为主要技术栈，可构建 H5，并保留微信小程序运行能力；后端采用 Spring Boot 3.5.13 和 Java 17；数据库采用 MySQL；实时消息采用 WebSocket/STOMP；上传文件保存在可外置的文件目录；AI 能力由后端封装外部兼容接口。

当前后端属于模块化单体，而不是已经拆分的微服务。仓库同时提供 Docker Compose 单机部署基线和 Kubernetes 清单、Kustomize 配置及发布脚本；是否已经完成公网主机、域名和 HTTPS 部署，仍以目标环境的实际验收结果为准。

1.3 文档概述

本文档说明松果集市的软件总体设计方案，主要包括：

- 系统总体架构和分层设计；
- 子系统与模块划分；

- 主要业务流程和接口关系；
- 数据架构与核心数据实体设计；
- 部署架构与运行环境设计；
- 性能、安全、可靠性、可维护性和扩展性设计；
- 设计验证、测试考虑、风险与未决事项。

本文以当前仓库代码、数据库脚本和部署配置为事实依据。对尚未实现的真实支付、售后、Redis 和微服务等内容，统一作为目标设计或未决事项说明，不作为当前能力。Kubernetes 清单已经进入主分支，但集群运行结果仍需由目标环境日志证明。

1.4 参考资料

- 《第 3 组 - 软件需求规格说明书》；
- 《第 3 组 - 软件开发计划书》；
- 项目当前源代码及 `README.md`；
- `shopping_back/shopping_back/doc/db.sql`；
- `deploy/docker-compose.yml`、Dockerfile 与 Nginx 配置；
- 项目部署文档、测试文档和需求追溯表；
- 《GB/T 8567-2006 计算机软件文档编制规范》；
- 《ISO/IEC 12207 软件生命周期过程》；
- 《软件工程基础 2026 夏季学期实践要求》；

1.5 术语与缩略语

术语/缩略语	说明
CSCI	计算机软件配置项，本项目可理解为松果集市系统及其主要软件模块
RBAC	基于角色的访问控制；当前实现为角色与业务规则相结合的简化权限控制
Token	登录后用于识别用户身份的随机字符串；当前并非 JWT
RESTful API	面向资源风格的 HTTP 接口设计方式
STOMP	系统 WebSocket 消息采用的应用层协议
AI Agent	基于大模型或规则的智能辅助模块
H5	移动端或桌面浏览器中的网页运行形态
SPA	单页应用，页面切换主要由前端路由完成

2 设计概述

2.1 设计目标

1. 建立满足校园新品与二手交易业务的总体软件结构，保证当前核心链路可运行、可测试。

- 2. 建立清晰的模块边界和接口关系，支持团队按模块并行开发。
- 3. 保证系统在课程项目资源约束下具备可部署、可演示、可维护和可扩展能力。
- 4. 兼顾平台治理、安全、性能和 AI 功能扩展，避免后期大范围返工。
- 5. 为详细设计、数据库设计、接口设计、测试设计和部署交付提供可落地的设计基线。
- 6. 清晰区分当前实现和后续目标，避免文档内容超前于代码与部署状态。

2.2 设计原则

- **分层设计**：界面层、接口层、业务层、数据层和智能服务职责分离。
- **核心优先**：优先保证注册登录、商品发布与审核、浏览、下单、评价和消息等核心能力。
- **安全默认**：对鉴权、权限校验、输入校验、敏感数据保护和管理操作进行安全设计。
- **配置优先**：数据库连接、允许源、上传目录和 AI 密钥通过环境配置管理。
- **可扩展**：AI、对象存储、缓存和消息能力通过清晰接口接入，不与核心交易链路强耦合。
- **易维护**：统一代码分层、接口响应、异常处理和部署方式。
- **渐进开发**：先稳定模块化单体，再依据性能和协作证据决定是否拆分服务。

2.3 运行环境

类别	设计要求
前端运行环境	Chrome、Edge 等现代浏览器；H5/uni-app 构建结果；微信开发者工具或微信运行环境
后端运行环境	Java 17、Spring Boot 3.5.13，默认端口 8080
数据库环境	MySQL 8.x；容器化基线使用 MySQL 8.4
Web 接入环境	Nginx 1.27 Alpine；以 <code>deploy/frontend.Dockerfile</code> 为容器化版本基线，提供静态资源和反向代理
开发环境	Windows 10/11、Maven、HBuilderX、VS Code 或其他代码编辑器
网络环境	支持本机、局域网和具备完整安全配置后的公网访问；公网必须启用 HTTPS/WSS

2.4 设计约束

- 后端核心采用 Java/Spring Boot，实现接口以 HTTP JSON 为主；
- 前端使用 uni-app，H5 构建产物由 Nginx 托管；
- 文档、代码、数据库、测试报告和部署结果必须保持一致；
- 项目需满足基本隐私保护、内容治理和网络安全要求；
- 当前容器构建依赖已生成的后端 JAR 和 H5 构建目录；
- 公网域名、证书、云主机和安全组不保存在仓库中，需要单独配置。

2.5 假设与依赖

- 系统初期面向课程演示和中小规模访问；
- 当前未接入真实支付和物流系统；
- 商品图片和资料图片首先使用服务器文件目录保存，并由持久卷保护；
- AI 模块依赖外部服务和密钥，但不得阻塞核心交易流程；
- 购物车和收货地址由后端 REST 接口与 MySQL 持久化，前端通过服务层访问；
- 当前没有 Redis 服务，缓存属于压测后按需要引入的增强方案。

3 系统总体设计

3.1 总体架构

松果集市由前端应用、Spring Boot 后端、MySQL 数据库、上传文件目录和外部 AI 服务组成。开发环境中，H5 开发服务器通常运行在 5173 端口并代理后端接口；容器化部署中，Nginx 统一提供页面、HTTP 接口、文件和 WebSocket 入口。

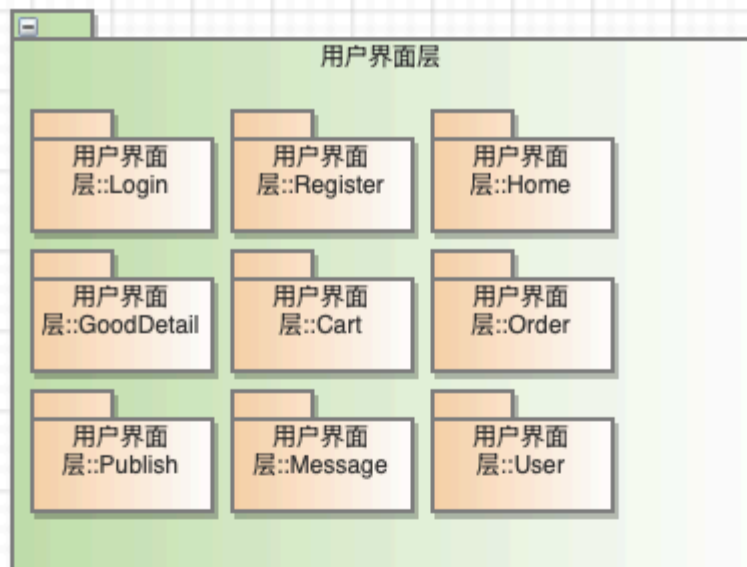
主要调用关系如下：

H5/小程序 → HTTP 或 WebSocket → Spring Boot → MyBatis Mapper → MySQL

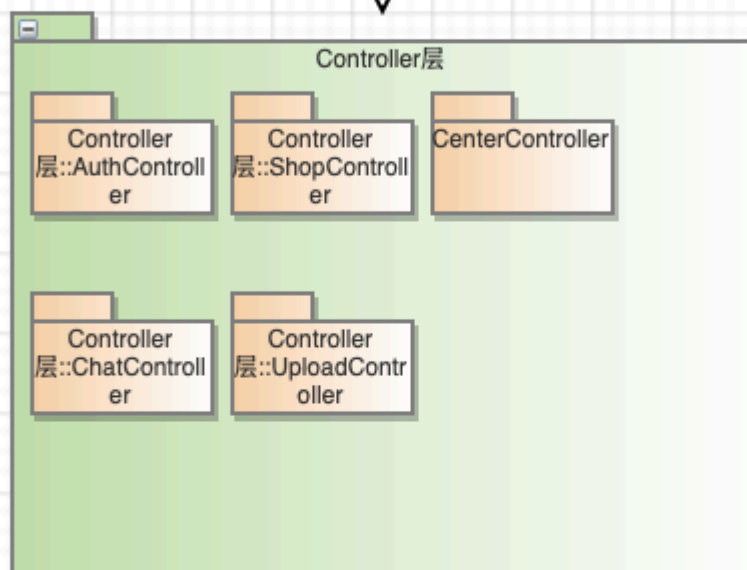
AI 调用关系如下：

客户端 → 后端 AI 服务 → 外部 AI 接口 → 后端降级或整理 → 客户端

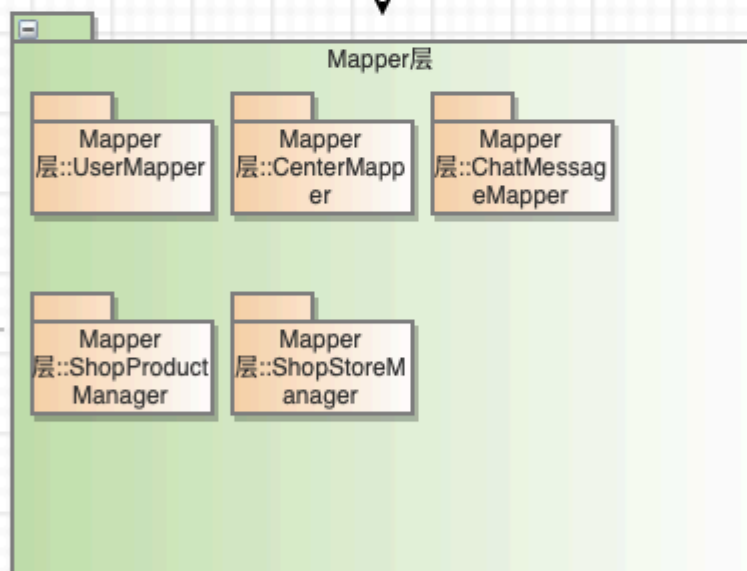
3.2 分层设计



请求接口



数据操作



映射实体



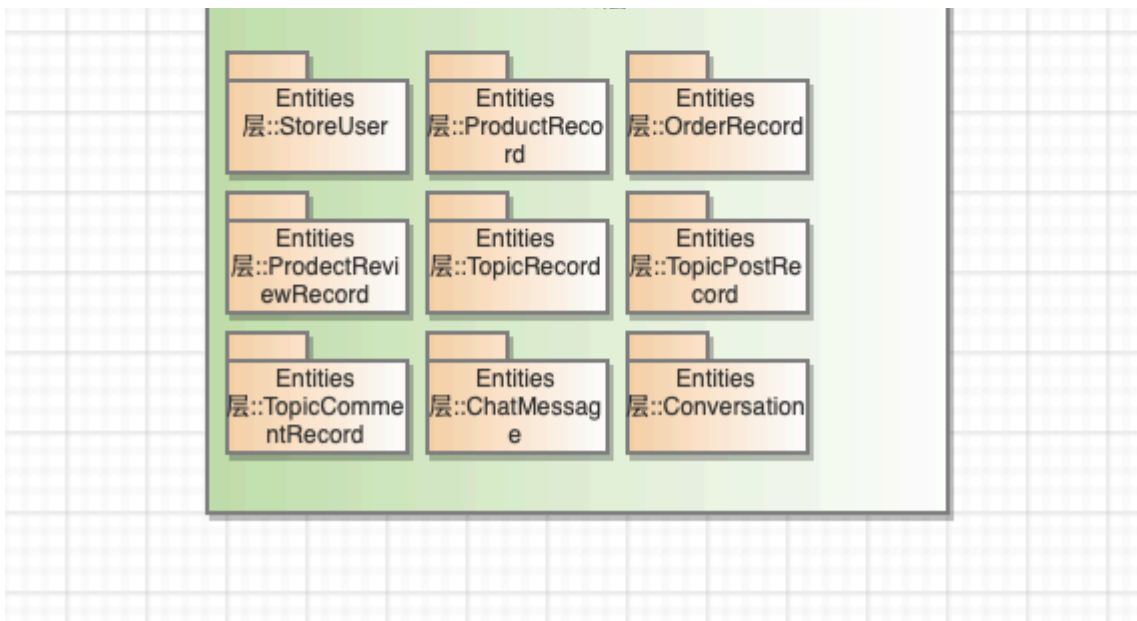


图 3-1 系统分层结构图

图 3-1 展示系统主要代码对象的分层关系，各层组成和职责如下。

层次	当前组成	职责说明
表现层	uni-app 页面、组件	页面渲染、表单交互、状态展示和移动端适配
前端服务层	services、utils、config	请求封装、登录态、购物车/地址接口适配和环境地址
接口层	Auth、Center、Shop、Cart、Address、Chat、Upload、AI Controller	参数绑定、协议转换、调用业务服务、统一返回
业务层	AuthService、CenterService、ShopService、CartService、AddressService、Chat/AI Service	业务规则、权限判断、交易与消息流程
数据访问层	User、Center、Product、Store、Order、Cart、Address、Topic、Chat Mapper	执行 SQL、对象映射和数据访问
数据层	MySQL、上传文件目录	保存业务数据和上传资源
接入层	Nginx	静态资源、反向代理、WebSocket 升级、限流和安全响应头

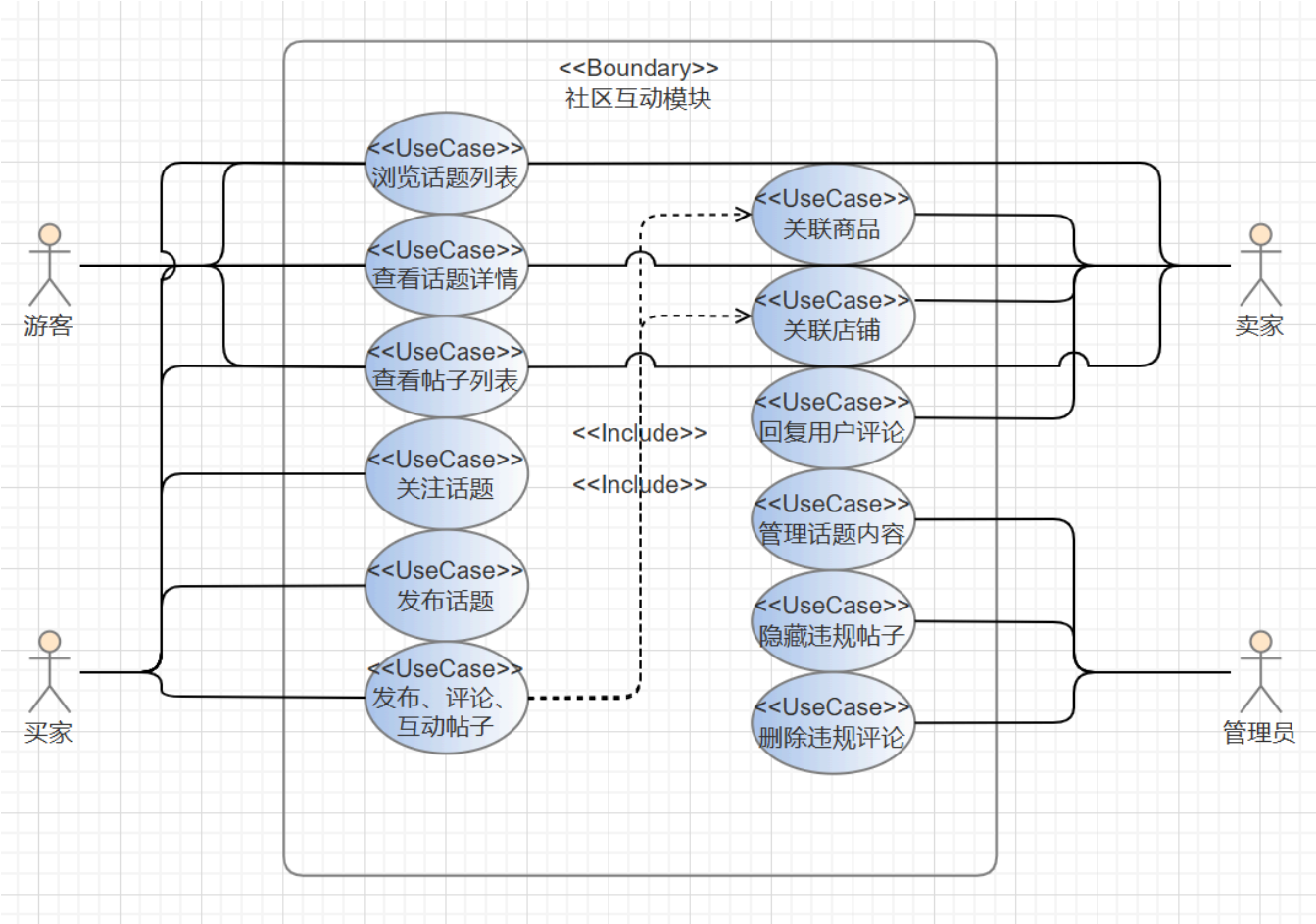
3.3 架构选型说明

1. 采用模块化单体而非微服务。团队规模和项目阶段更适合统一构建、联调和部署，同时通过 `auth`、`center`、`shop`、`chat`、`config` 和 `common` 包保持边界。
2. 采用 MySQL 作为主库。当前数据结构化程度高，订单、评价、实名和信用等业务需要事务支持。
3. 当前不使用 Redis。仓库依赖和 Compose 服务中均无 Redis。缓存组件仅在性能测试确认存在热点瓶颈后纳入设计变更。
4. AI 模块独立接入。AI 能力存在外部依赖、延迟和费用风险，必须能够超时和降级。

5. 当前使用随机 Token 鉴权。Token 保存在 AuthService 的内存映射中，不是 JWT。生产部署前必须补充过期、撤销和多实例共享机制。

3.4 角色与权限模型

角色	主要权限
访客	浏览公开商品、店铺和社区内容，进入注册登录页面
普通用户	资料维护、收藏关注、购物车、下单、评价、聊天和实名认证
卖家相关用户	发布和管理本人商品、查看卖家中心、参与订单和聊天
管理员	用户状态与信用管理、实名认证、商品审核



3.5 总体业务流程

1. 用户注册或登录，后端验证 BCrypt 密码并签发随机 Token。
2. 用户维护资料，并可提交买家或卖家实名认证。
3. 用户浏览、搜索和筛选商品，执行收藏、关注和浏览记录操作。
4. 卖家发布商品，管理员通过商品审核接口进行审核。
5. 买家通过服务端购物车选择商品，从本人服务端地址中选择收货地址，再提交后端创建订单并查看订单。
6. 交易双方可通过站内会话和 WebSocket 沟通，并可使用 AI 议价建议。
7. 用户提交商品评价，系统保存评价并联动信用记录。

8. 管理员执行用户、信用、实名和商品审核管理。

4 子系统与模块设计

4.1 模块划分

原有八个业务模块继续保留，但按照当前代码标注实现状态。

模块编号	模块名称	说明	当前状态
M1	用户与权限模块	注册登录、资料、实名、角色、地址与信用	主要能力已实现；Token 生产化仍需完善
M2	商品与内容模块	商品、店铺、媒体资源和商品审核	主要能力已实现
M3	搜索与推荐模块	关键词、分类、排序和推荐展示	基础查询已实现；高级推荐待完善
M4	购物车与订单模块	购物车/地址接口、订单与金额	服务端接口和数据表已实现，但当前结算页仍使用本地购物车/地址；订单状态仍不完整
M5	售后与评价模块	评价、信用、退款与售后	评价与信用已实现；售后退款未实现
M6	平台治理模块	实名审核、商品审核、用户治理	部分实现；举报仲裁和完整审计待实现
M7	AI 扩展模块	议价、发布建议和购物助手	已接入，可降级，依赖外部服务
M8	基础支撑模块	文件、消息、异常、跨域、部署	主要基础能力已实现；监控待完善

4.2 用户与权限模块设计

4.2.1 模块职责

- 提供用户注册、登录、当前用户查询和资料修改；
- 管理买家/卖家实名认证申请及管理员审核；
- 管理用户状态和信用记录；
- 为商品、订单、评价、消息和治理模块提供身份信息。

4.2.2 主要子功能

子功能	当前设计
注册登录	支持用户名和密码注册登录，密码使用 BCrypt 保存

子功能	当前设计
用户资料	支持查询和更新当前用户资料
实名认证	支持提交、撤销、管理员通过和拒绝
信用管理	使用用户信用汇总和 <code>credit_record</code> 记录变化
地址管理	<code>/api/address</code> 提供本人地址增删改查和默认地址切换，数据保存在 <code>user_address</code>
权限控制	使用 Token、管理员身份和资源归属执行简化权限判断

4.2.3 对外接口

主要通过 `/api/auth` 和 `/api/center` 提供注册、登录、当前用户、角色中心、实名和管理接口。当前 Token 存在后端内存中，应用重启后登录态失效，也不适合直接扩展多个后端实例。

4.3 商品与内容模块设计

4.3.1 模块职责

- 管理商品查询、发布、编辑、上下架和删除；
- 管理商品价格、成色、分类、图片和交易描述；
- 提供店铺展示、收藏、关注和浏览历史；
- 对接管理员商品审核；
- 为首页、搜索、订单和消息提供商品信息。

4.3.2 商品状态设计

原设计中的完整商品状态机作为目标保留：

状态	说明
DRAFT	草稿
PENDING_REVIEW	待审核
REJECTED	审核驳回
ON_SALE	上架中
LOCKED	已锁定，进入交易占用
SOLD	已售出
OFF_SHELF	已下架
DELETED	逻辑删除

当前实现已经具备商品发布、查询、维护和审核相关能力，但状态字段值及流转尚未完全统一为上表。后续详细设计应从实际数据库字段和服务方法建立唯一状态枚举，防止文档、前端和数据库各自使用不同名称。

4.3.3 设计要点

- 商品发布和编辑应进行参数、图片和权限校验；
- 商品列表支持筛选和排序，并对结果数量进行限制；
- 商品可见性由服务端审核状态决定；
- 商品修改、删除和审核必须验证操作者权限；
- 后续补齐订单状态时，商品可售状态应与订单事务一致。

4.4 搜索与推荐模块设计

4.4.1 模块职责

- 提供商品关键词、分类和条件筛选；
- 为首页精选、新品、二手、信用优先和价格优先等展示提供数据；
- 记录浏览行为，为后续个性化推荐提供基础。

4.4.2 设计要点

- 当前以数据库条件查询和规则排序为主；
- 分页、索引和 SQL 优化优先于引入缓存；
- 推荐结果不足时应显示真实空状态，不使用虚构商品数据；
- 是否引入 Redis 必须由压力测试和慢查询结果决定。

4.5 购物车与订单模块设计

4.5.1 模块职责

- 管理购物车商品选择和结算信息；
- 管理订单创建、订单查询和金额记录；
- 向评价和信用模块提供订单基础数据。

4.5.2 当前实现

- 服务端购物车由 `CartController`、`CartService`、`CartMapper` 提供列表、添加、改量、勾选和删除接口，写入 `cart_item`，但当前结算页尚未调用这些接口；
- 服务端地址由 `AddressController`、`AddressService`、`AddressMapper` 提供增删改查和默认地址切换，写入 `user_address`，但当前结算页仍读取前端本地地址；
- 订单写入 MySQL 的 `orders` 表；
- 当前创建订单后直接写入 `completed` 状态；
- 当前订单请求只包含商品编号与数量，不包含地址快照；后端不清理服务端购物车，也没有多商品统一事务；没有真实支付和物流接口。

4.5.3 目标订单状态设计

状态	说明
PENDING_PAYMENT	待支付
PAID	已支付待发货
SHIPPED	已发货
RECEIVED	已确认收货
COMPLETED	已完成
CANCELLED	已取消
AFTER_SALE	售后处理中
REFUNDED	已退款

该状态表为后续目标，不代表当前全部实现。补齐时必须同步设计状态转换条件、操作者权限、库存/商品状态、幂等规则、事务和测试用例。

4.6 售后与评价模块设计

4.6.1 当前能力

- 用户可提交商品评价；
- 评价保存在 `product_review`；
- 评价结果可更新卖家或店铺信用，并形成 `credit_record`。

4.6.2 待实现能力

- 退款、退货、投诉和售后工单；
- 卖家响应、证据附件和管理员仲裁；
- 售后状态机、退款记录和审计记录。

在数据表、接口和测试未完成前，不把上述待实现能力列入当前验收范围。

4.7 平台治理模块设计

4.7.1 当前能力

- 用户状态和信用管理；
- 买家/卖家实名认证审核；
- 商品审核。

4.7.2 后续能力

- 举报受理、申诉复核和争议仲裁；
- 通用治理记录和操作审计；

- 敏感内容检测和规则配置；
- 高风险管理操作的二次确认。

4.8 AI 扩展模块设计

4.8.1 模块职责

- 提供 AI 议价建议；
- 提供商品发布建议；
- 提供购物助手问答；
- 统一封装外部 AI 服务，避免前端接触密钥。

4.8.2 设计要点

- AI 调用与交易主流程隔离；
- API 密钥通过环境变量注入；
- 对输入长度和必要隐私信息进行控制；
- 外部服务失败时返回可理解的降级结果；
- 压力测试使用 Mock、限流或受控账户，避免产生不可控费用。

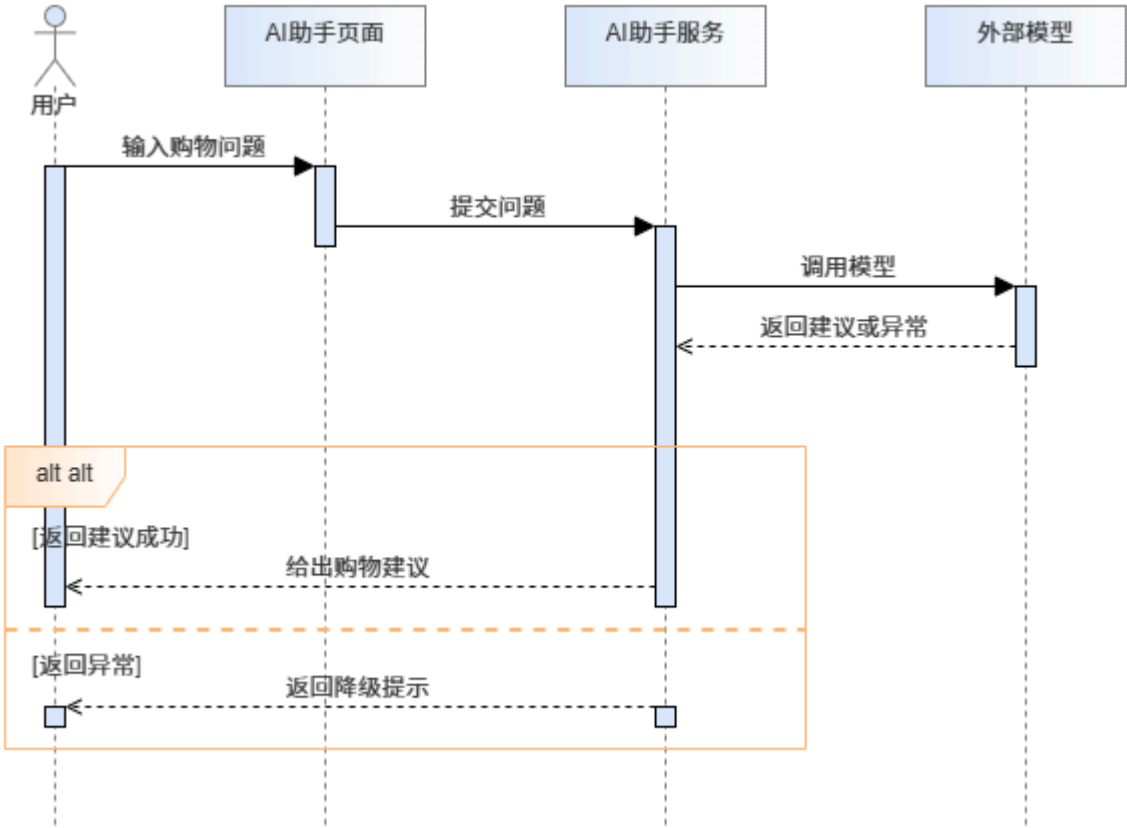


图 4-8 AI 购物助手组件级顺序图

用户通过前端 AI 助手组件输入购物问题；AI 助手组件向 AI 服务组件提交请求；AI 服务组件调用外部模型组件获取回答，成功时返回整理后的建议，失败时返回降级提示。

4.9 基础支撑模块设计

4.9.1 模块职责

- 文件上传及 `/files/**` 资源访问;
- 统一响应 `ApiResponse`;
- 全局异常处理;
- CORS 与运行环境配置;
- WebSocket/STOMP 实时消息;
- Docker 健康检查和 Nginx 接入。

4.9.2 设计要点

- 文件上传校验类型、大小、文件名和路径;
- 生产异常响应不泄露 SQL、密钥和堆栈;
- WebSocket 连接必须校验 Token;
- 生产环境允许源由 `PUBLIC_ORIGIN` 控制;
- 上传目录必须挂载持久卷并纳入备份。

4.10 业务用例组件级动态模型整理

4.10.1 UC-01 注册并登录

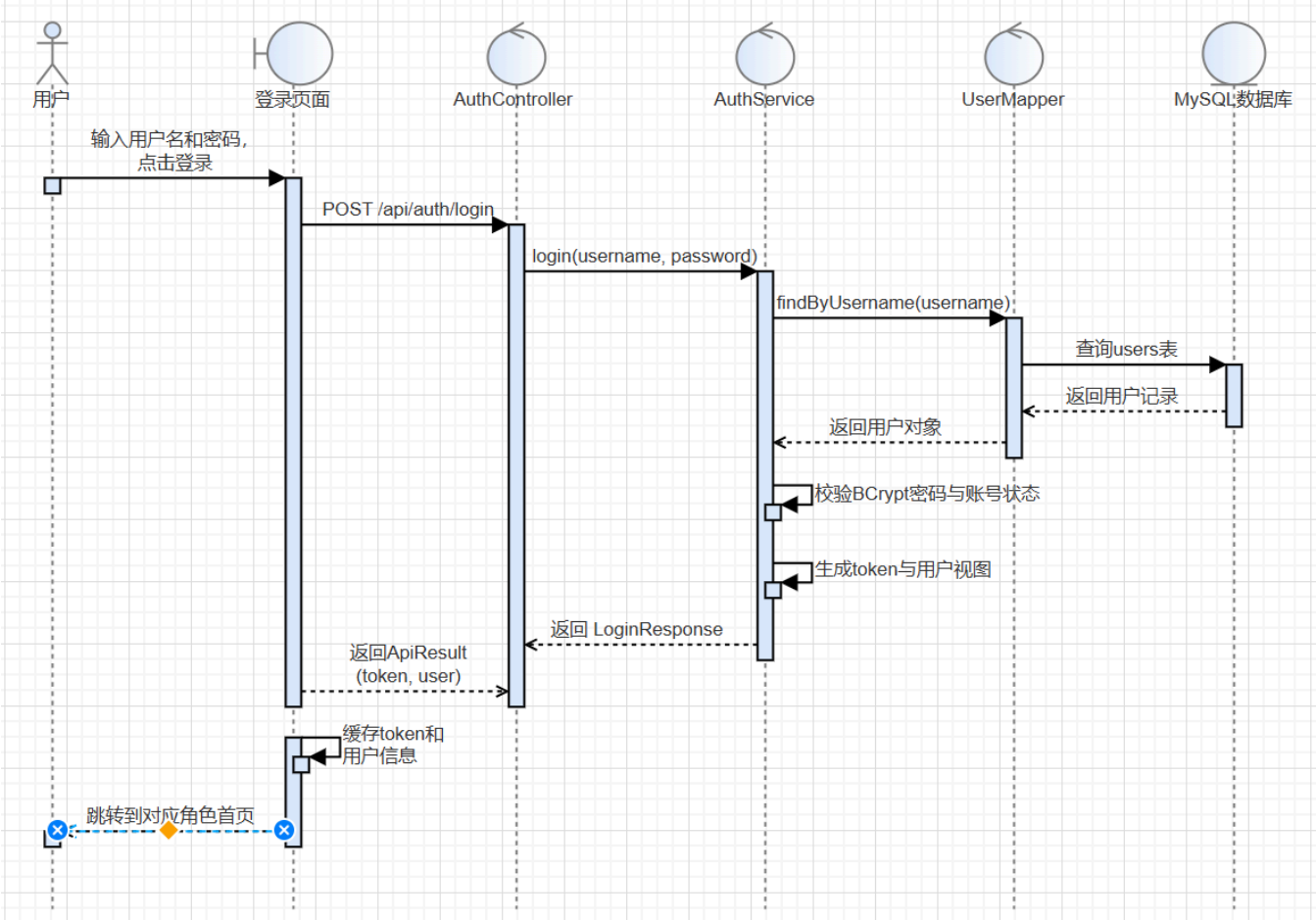


图 4-9 UC-01 注册并登录组件级顺序图

4.10.2 UC-06 卖家发布并维护商品

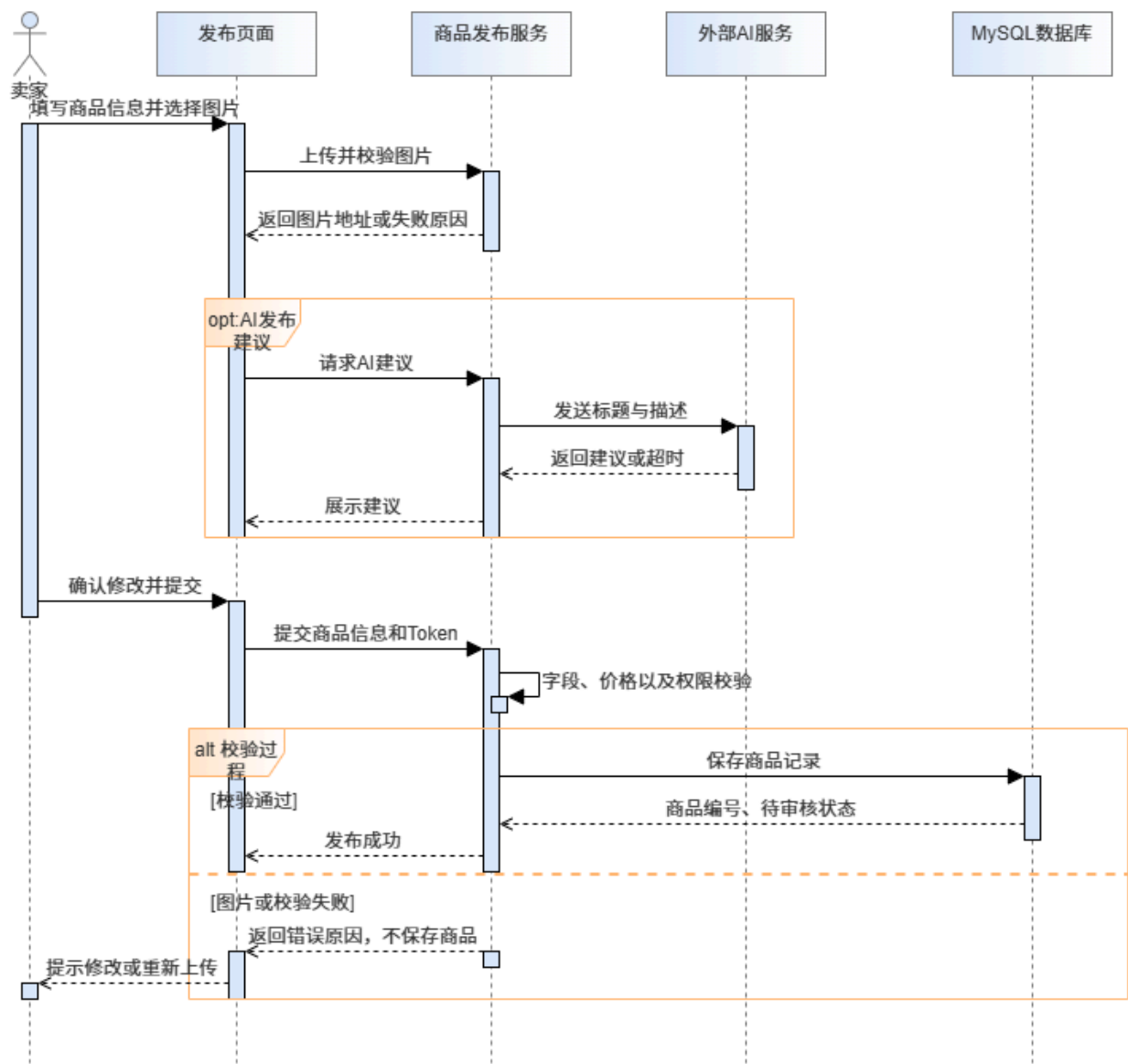


图 4-10 UC-06 卖家发布并维护商品组件级顺序图

4.10.3 UC-07 管理员审核商品

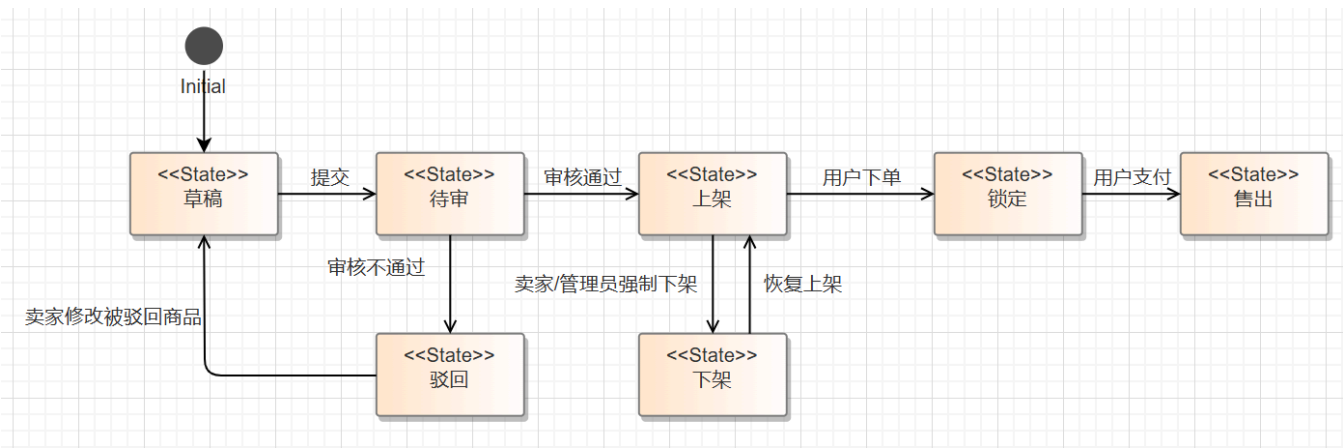


图 4-11 UC-07 管理员审核商品组件级顺序图

4.10.4 UC-08 维护购物车和地址并创建订单

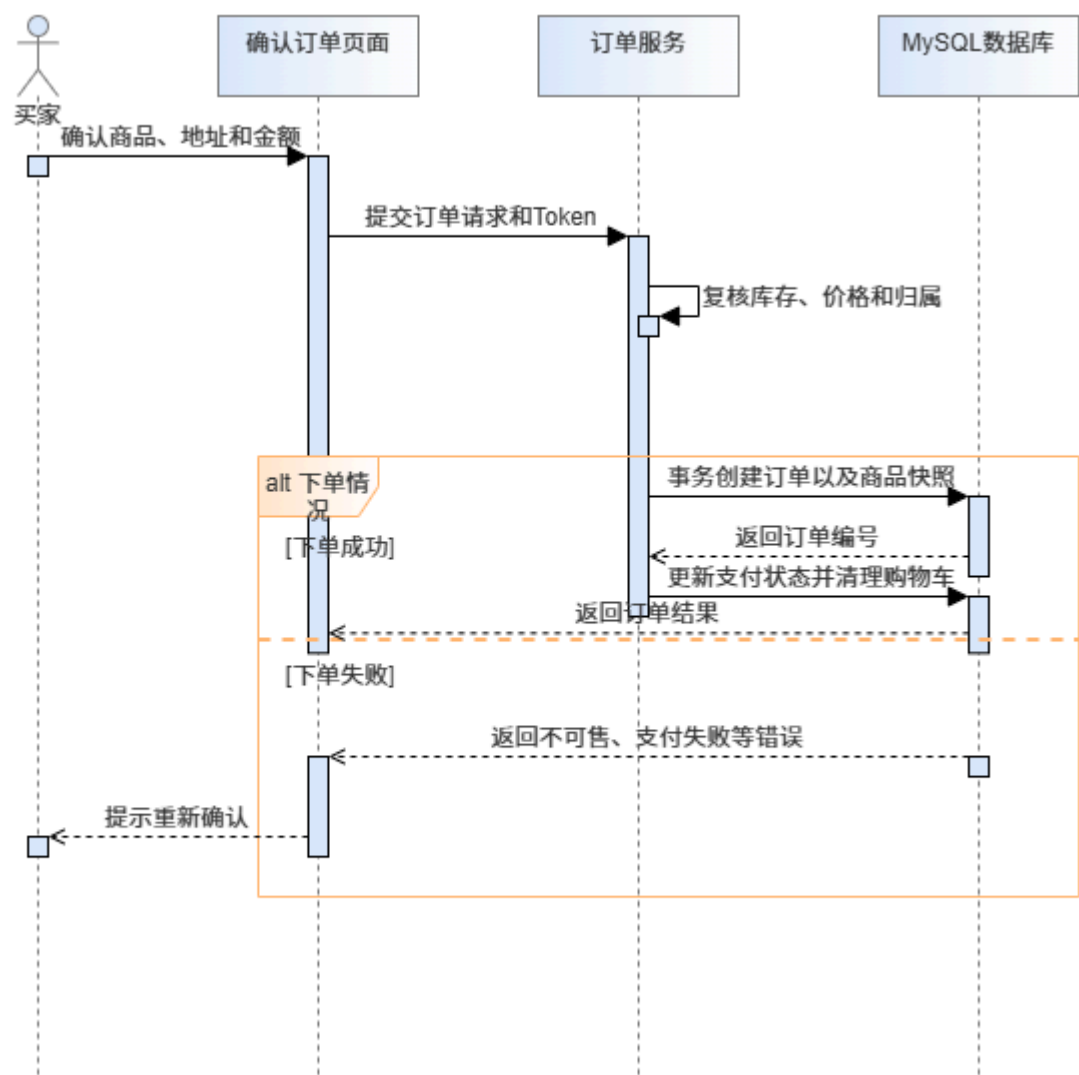


图 4-12 UC-08 维护购物车和地址并创建订单组件级顺序图

4.10.5 UC-09 查询订单并提交评价

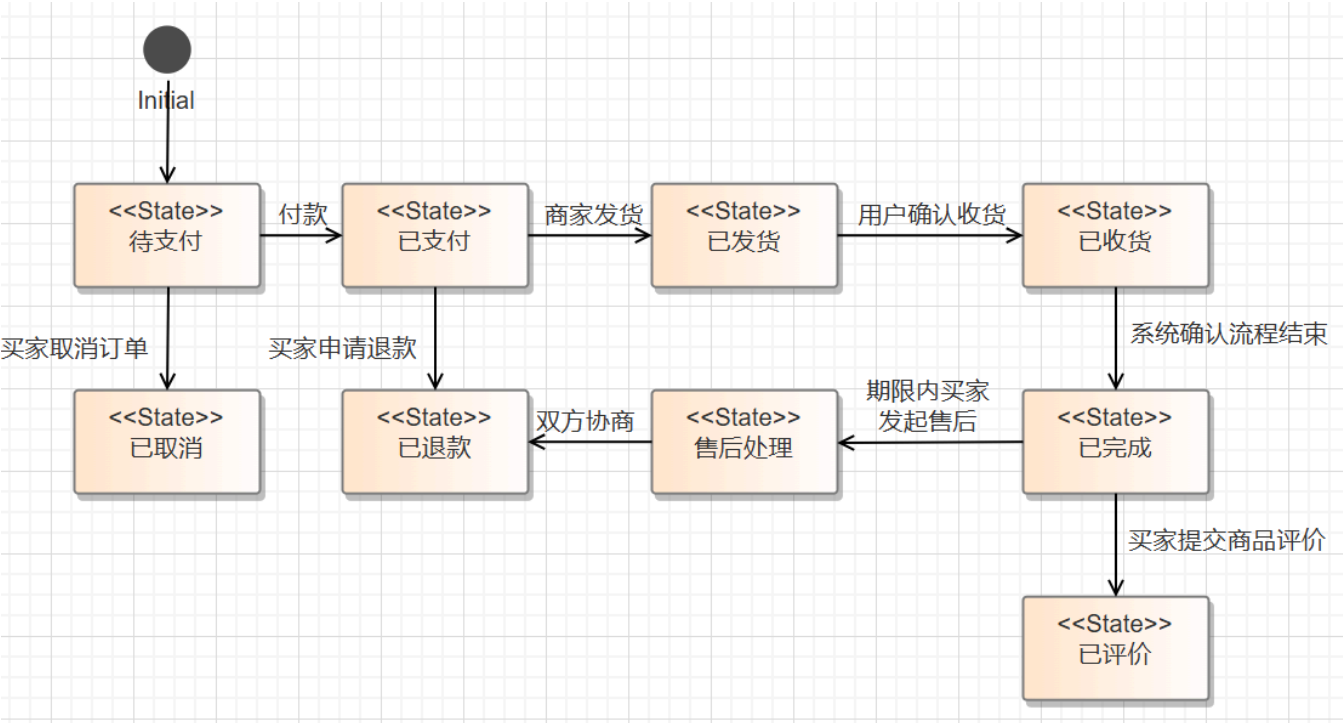


图 4-13 UC-09 查询订单并提交评价组件级顺序图

4.10.6 UC-10 创建话题并完成社区互动

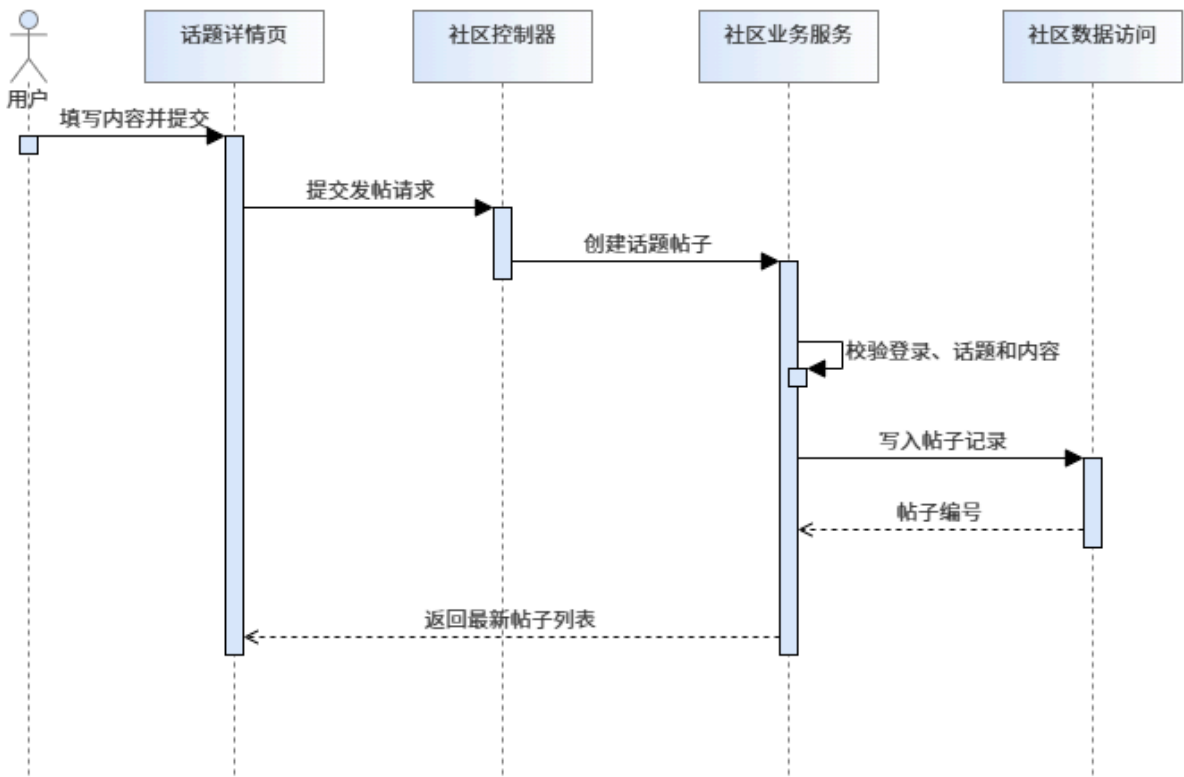


图 4-14 UC-10 创建话题并完成社区互动组件级顺序图

4.10.7 UC-11 建立会话、收发消息并使用 AI 议价

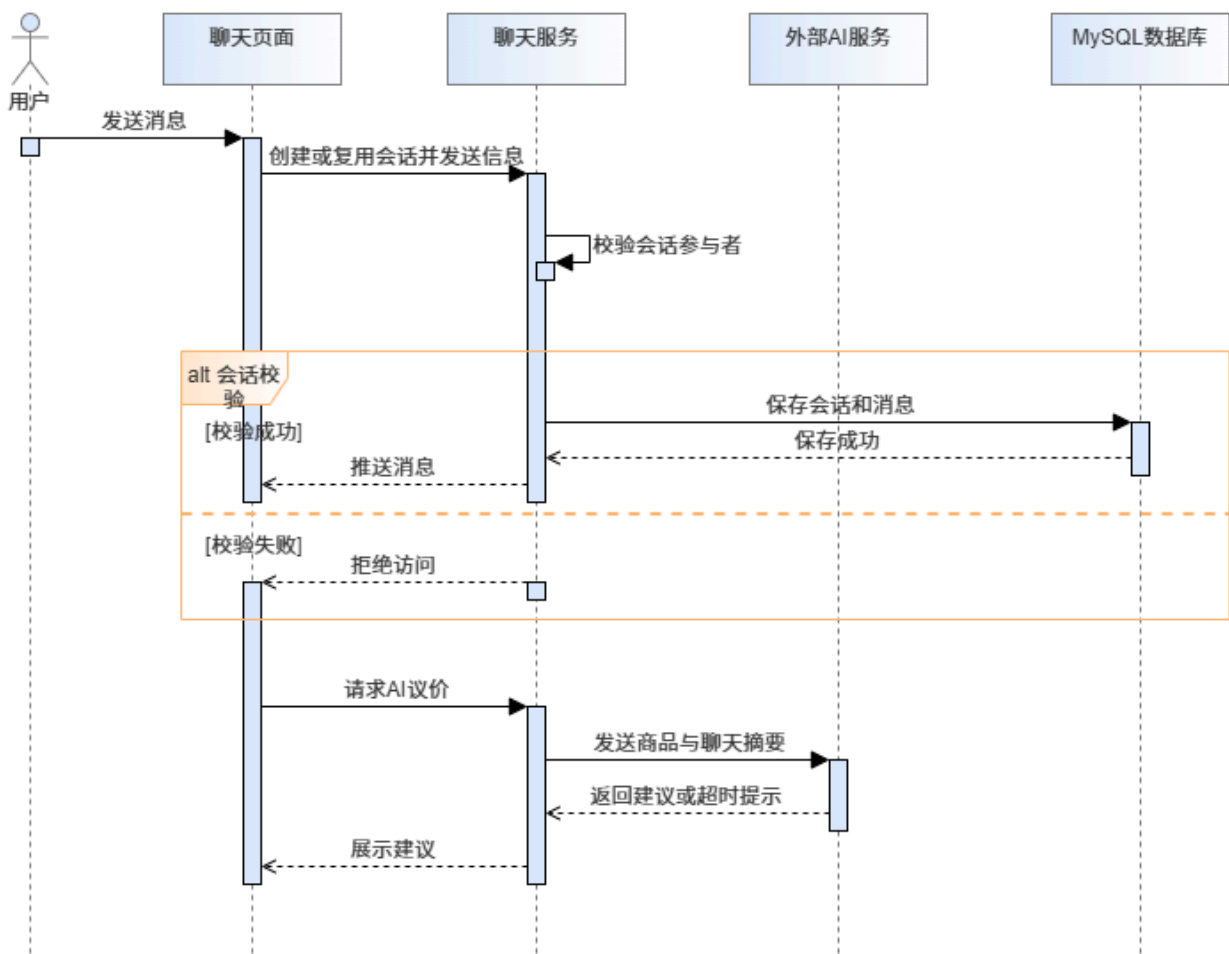


图 4-15 UC-11 建立会话、收发消息并使用 AI 议价组件级顺序图

4.10.8 UC-12 提交实名认证并由管理员审核

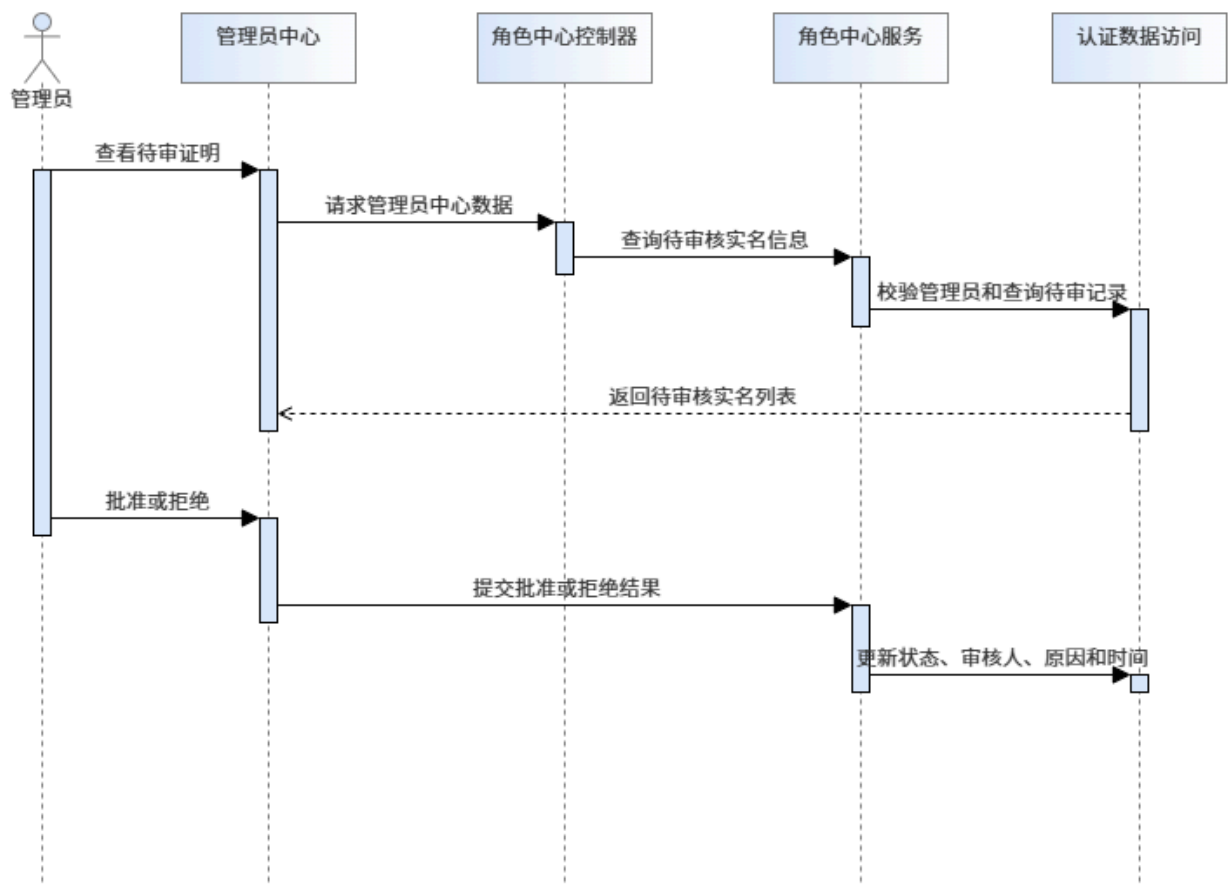


图 4-16 UC-12 提交实名认证并由管理员审核组件级顺序图

4.10.9 UC-13 使用 AI 购物助手

现有组件级顺序图见 4.8 节图 4-8。本节只建立用例映射，不重复插入和编号同一张图片。

4.10.10 当前未覆盖项

用例	现有相关图片	组件级动态模型判定
UC-02 浏览、搜索并查看商品	商品发现与收藏活动图	系统业务活动图，不是组件级模型；未独立覆盖
UC-03 收藏商品、关注店铺并查看	商品发现与收藏活动图、用户店铺关系图	前者为系统级、后者为静态关系图；未独立覆盖
UC-04 维护个人资料与头像	登录顺序图、用户店铺关系图	场景不一致；未覆盖
UC-05 卖家查看并维护店铺	商品发布顺序图、用户店铺关系图	只间接涉及卖家/店铺；未独立覆盖
UC-14 管理员维护用户状态与信用	用户账号治理状态机图	该图放在详细设计并描述对象状态，不能当作组件协作图

5 接口概要设计

5.1 外部接口设计

接口类别	说明
用户接口	H5 页面、微信小程序页面和管理页面
软件接口	MySQL、上传文件存储、外部 AI 服务
通信接口	HTTP/HTTPS、JSON、WebSocket/STOMP
硬件接口	通用服务器、PC 和手机，无专用硬件依赖

5.2 API 设计规范

5.2.1 请求与响应规范

- 协议：开发环境可使用 HTTP，生产环境使用 HTTPS；
- 数据格式：JSON；
- 字符编码：UTF-8；
- 鉴权方式：当前为随机 Token，不是 JWT；
- 接口前缀： /api ；
- 统一响应： ApiResult ，包含业务码、提示和数据；
- 错误响应同时使用合适的 HTTP 状态码。

5.2.2 通用状态码

状态码	说明
200	请求成功
400	参数错误
401	未登录或 Token 无效
403	无权限
404	资源不存在
409	状态冲突或重复操作
500	服务器内部错误

5.3 主要接口分组

接口分组	当前代表路径	说明
用户接口	/api/auth/register、/api/auth/login、/api/auth/me	注册、登录和当前用户
用户中心	/api/center/buyer、/api/center/seller、/api/center/admin	用户中心、实名和管理
商品接口	/api/products	商品查询、发布、维护
店铺接口	/api/stores	店铺与关注
社区接口	/api/topics、/api/topic-posts	话题、帖子和互动
订单接口	/api/orders	订单创建、查询与评价相关操作
审核接口	/api/admin/audit	商品审核
聊天接口	/api/chat	会话、消息、状态和 AI 议价
实时消息	/ws	WebSocket/STOMP 连接
AI 接口	/api/ai/assist、/api/ai/publish-suggestion、/api/ai/assistant	AI 辅助能力
文件接口	/api/upload/image、/files/**	上传和读取文件

5.4 内部接口设计原则

- Controller 通过 Service 执行业务，不跨层直接操作数据库；
- Service 负责权限、状态和事务边界；
- Mapper 只负责 SQL 与数据映射；
- 模块之间通过明确的 DTO、方法和业务对象交互；
- 外部 AI 调用必须设置超时、异常处理和降级；
- 实时消息和 HTTP 消息历史共同使用持久化数据，避免只推送不落库。

6 数据概要设计

6.1 数据架构概述

当前数据分为以下四类：

- 用户与治理数据：用户、实名、信用；

- 商品与行为数据：商品、店铺、收藏、浏览、关注；
- 交易与消息数据：订单、评价、会话、聊天消息；
- 社区数据：话题、帖子、评论、点赞和动作记录。

6.2 核心实体设计

数据域	当前表
用户与认证	users、user_realname_auth
商品与店铺	goods、store
用户行为	favorite_goods、browse_history、follow_store、follow_topic
交易与信用	orders、product_review、credit_record
消息	conversation、chat_message
社区	community_topic、topic_post、topic_comment、topic_post_like、topic_post_action

6.3 数据关系说明

1. 一个用户可发布多个商品，并产生收藏、浏览、关注、订单、评价和信用记录。
2. 商品通过 seller_id 归属卖家；店铺同样通过 seller_id 归属卖家。商品可被收藏、浏览、下单和评价。
3. 订单关联买家与商品信息；当前表结构和业务尚未实现多订单项的完整快照模型。
4. 一个会话包含多条聊天消息，消息保存发送者、接收者、内容、时间和状态。
5. 一个社区话题包含多个帖子，帖子包含评论、点赞和动作记录。

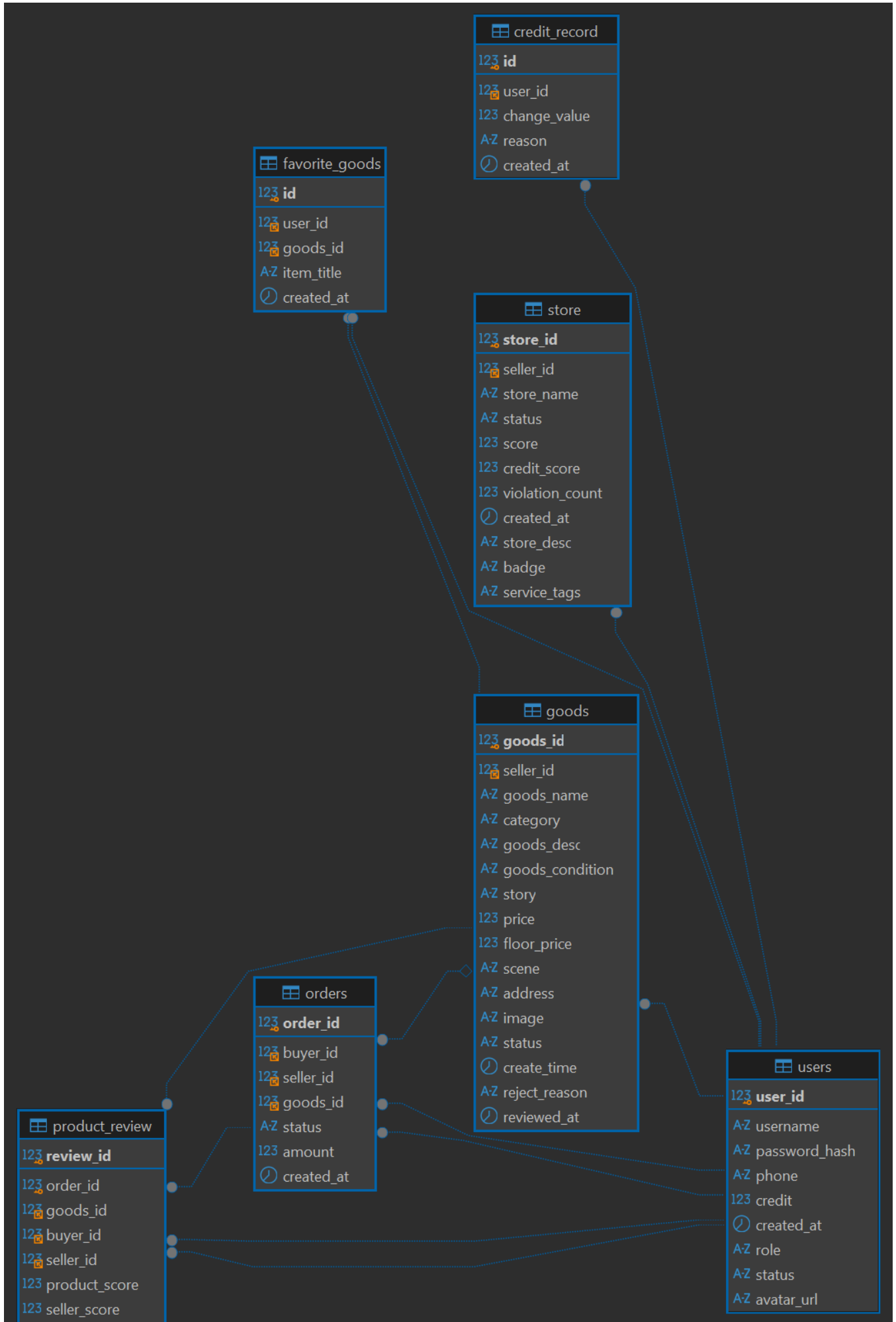


图 6-1 校园二手交易平台核心交易 ER 图

6.4 数据库设计原则

- 表和字段使用统一命名；
- 重要记录包含必要的创建时间、更新时间和状态；
- 用户、商品、订单、会话、状态和时间等高频条件建立适当索引；
- 多表写操作由 Service 事务控制；
- 收藏、关注、点赞等操作通过约束或逻辑校验保证幂等；
- 初始化以 `doc/db.sql` 为准；数据库增量变更统一使用 Flyway 或 Liquibase 管理；
- 运行期散落在 Mapper 中的兼容 DDL 应逐步收敛，避免环境结构漂移。

6.5 缓存设计

当前系统没有接入 Redis，数据库和上传目录是主要持久化存储，Token 使用进程内映射。缓存设计作为后续性能增强保留：

- 缓存只作为性能增强，不作为唯一数据源；
- 先通过 JMeter、慢查询和指标确认首页、商品或会话热点；
- 引入缓存后统一键命名、过期时间和失效策略；
- 登录态若迁移到 Redis，必须同时设计过期、撤销、续期和故障行为。

7 关键设计说明

7.1 安全设计

7.1.1 身份认证与权限控制

- 密码使用 BCrypt 哈希保存；
- 受保护接口校验 Token、角色和资源归属；
- 管理操作必须由后端验证管理员身份；
- 当前内存 Token 无明确过期和多实例共享，生产前需要整改；
- 使用统一鉴权拦截器实施接口身份与权限校验。

7.1.2 输入与内容安全

- 对请求参数执行前后端校验；
- 对上传文件检查大小、类型、扩展名和安全文件名；
- 对商品、评论和聊天等文本逐步补充内容安全策略；
- 生产错误响应不得返回 SQL、绝对路径、堆栈和秘密信息。

7.1.3 数据与隐私保护

- 实名、联系方式和地址等敏感信息遵循最小可见和脱敏原则；

- 数据库密码和 AI 密钥通过 `.env` 或 CodeArts 秘密变量注入；
- 测试数据不得直接复制真实敏感数据；
- 数据库和上传文件需要备份及恢复验证。

7.2 性能设计

指标项	验收指标
注册/登录平均响应时间	常规条件下不高于 1 秒
商品列表平均响应时间	常规条件下不高于 800 ms
商品详情页面加载时间	常规网络下不高于 2 秒
创建订单平均响应时间	常规条件下不高于 1 秒
AI 单次响应时间	不高于 5 秒，超时必须降级
并发能力	以 JMeter 测试 200 并发为阶段性目标，并记录 P95/P99 与错误率

性能设计采用分页查询、必要索引、静态资源缓存、gzip 和 Nginx 基础限流。是否加入 Redis、异步队列或拆分服务，由压力测试、CPU/内存、数据库慢查询和调用链数据决定。

7.3 可靠性设计

- 关键写操作保证事务性，逐步为订单等接口补充幂等控制；
- AI 等非核心模块异常时不影响商品和订单基础接口；
- MySQL 和上传文件使用独立持久卷；
- MySQL、后端和前端均配置容器健康检查；
- 完成数据库与文件备份、恢复和上一镜像回滚演练；
- 当前单机 Compose 存在主机单点，不宣称高可用。

7.4 可维护性设计

- 统一 Controller、Service、Mapper 分层；
- 统一接口响应和异常处理；
- 使用配置文件和环境变量管理环境差异；
- 数据库变更集中为可审查的迁移脚本；
- 代码变更关联 CodeArts 工作项、需求和测试；
- 文档中的接口、表和部署状态随实现同步更新。

7.5 可扩展性设计

- 用户、商品、交易、消息/AI 和社区可作为未来候选服务边界；
- 在拆分前先稳定模块接口、数据归属、自动化测试和部署流水线；
- AI、对象存储、缓存和监控通过适配接口接入；

- 微服务和消息队列必须由真实扩缩容或协作需求驱动；现有 Kubernetes 清单用于部署验证，不代表系统已经拆分为微服务。

8 部署概要设计

8.1 部署架构

系统支持 Docker Compose 单机容器化部署和 Kubernetes 集群部署，部署组件和调用关系如图 8-1 所示。两种方式均不包含 Redis。

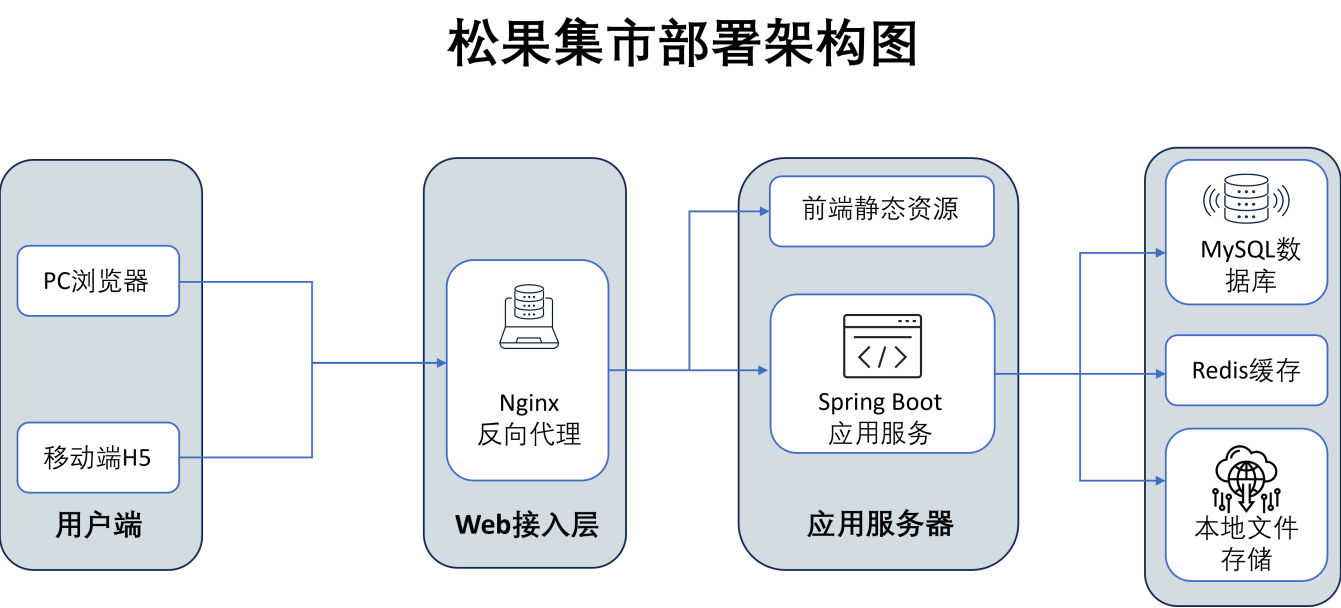


图 8-1 部署架构图

当前仓库的实际单机容器化部署组件如下：

组件	当前部署说明
前端	Node 18-alpine 构建 H5 静态资源，由 nginx:1.27-alpine 托管
后端	maven:3-eclipse-temurin-17 执行 mvn -DskipTests package，运行于 eclipse-temurin:17-jre-alpine，使用非 root 用户
数据库	mysql:8.4，使用 mysql_data 持久卷和 db.sql 初始化
文件存储	后端 /app/uploads 挂载 upload_data 持久卷，通过 /files/** 访问
外部服务	AI 接口由后端访问，密钥通过环境变量提供
缓存	当前没有 Redis 容器或后端 Redis 依赖

8.2 部署方式

Docker Compose 编排 frontend、backend 和 mysql 三个服务。只有 frontend/Nginx 端口映射到宿主机；backend 的 8080 和 mysql 的 3306 仅在内部网络访问。集群环境通过 `k8s/` 下的 Namespace、ConfigMap、MySQL、backend、frontend 和 Kustomize 清单部署，并由 `scripts/deploy-k8s.*` 执行发布。

Nginx 提供：

- H5 静态资源和 SPA 路由回退；
- `/api/` 到 backend 的反向代理；
- `/files/` 到上传资源的反向代理；
- `/ws/` 的 WebSocket 升级代理；
- 30 次/秒的基础 API 限流和 60 次突发；
- 10 MB 请求体限制、gzip、静态缓存和基础安全响应头。

当前 Dockerfile 使用多阶段构建，在镜像构建阶段完成后端 JAR 打包和前端 H5 构建；后端镜像构建明确使用 `-DskipTests`，单元测试由流水线前序独立阶段执行，本机无需预先生成制品。

8.3 环境划分

环境	用途	当前说明
开发环境	本地开发与联调	H5 通常使用 5173，后端使用 8080，MySQL 本地或容器运行
测试环境	集成、回归、压力和安全测试	应使用独立数据库和 AI Mock/受控密钥
演示/验收环境	课程答辩和最终展示	使用局域网目标主机或独立服务器，固定配置并留存报告
生产环境	公网正式服务	当前仓库未证明已经具备云主机、域名、DNS 和 HTTPS，应单独实施与验收

8.4 部署与运维要求

- 使用 `.env` 或 CodeArts 秘密变量提供数据库密码、允许源和 AI 密钥；
- `.env` 不提交仓库，生产密码不得使用示例值；
- MySQL 健康后启动 backend，backend 健康后启动 frontend；
- 后端当前使用 `/api/products` 作为健康检查；部署完善时增加专用就绪探针；
- 生产环境配置域名、HTTPS 和 WSS，HTTP 重定向 HTTPS；
- 当前消息页面 WebSocket 地址硬编码为 `http://127.0.0.1:8080/ws`，正式部署前必须改为同源 `/ws` 或环境地址；
- 备份 `mysql_data` 和 `upload_data`，并执行恢复演练；
- CodeArts 流水线依次执行测试、构建、制品保存、测试部署、验收和生产审批；
- 发布失败时使用上一可用镜像、配置和数据库恢复方案回滚。

9 设计验证与测试考虑

9.1 设计验证重点

- 当前核心业务链路是否可运行；
- 角色权限和数据归属是否由后端正确校验；
- 商品审核、订单、评价和信用状态是否一致；
- WebSocket 在局域网、反向代理和 HTTPS/WSS 环境下是否可连接；
- 页面、接口、数据库、部署配置和文档是否一致；
- 性能和安全指标是否由真实报告支持；
- 容器重启后数据库与上传文件是否保留；
- 备份恢复和部署回滚是否可执行。

9.2 测试要求

测试层次	工具/方式	验证重点
单元测试	JUnit	AuthService 和核心服务的正常、异常、权限与边界分支
白箱覆盖	JaCoCo	行覆盖、分支覆盖及核心服务未覆盖路径
接口黑箱	Postman/Newman 或冒烟脚本	注册、登录、商品、实名、订单、审核、聊天和 AI 接口
UI 自动化	Selenium	登录、商品浏览发布、购物车/地址/订单和管理审核流程
集成测试	测试环境	前端、后端、数据库、文件和 WebSocket 联调
压力测试	JMeter	吞吐量、P95/P99、错误率、CPU、内存和数据库瓶颈
安全测试	OWASP ZAP 与人工越权测试	注入、XSS、鉴权、上传、敏感数据和错误配置
部署测试	Docker 健康检查、备份与回滚演练	启停顺序、持久化、故障恢复和制品回退

截至 2026-08-29，后端单元测试已在本机实测 79 项，其中 78 项通过、1 项跳过、无失败和错误。主分支包含 66 个 Postman/Newman 请求级用例和 14 个 Selenium 场景，接口与端到端测试已完成历史执行。

10 设计风险与未决事项

10.1 设计风险

优先级	风险项	风险说明	应对策略
P0	WebSocket 地址硬编码	远程浏览器会连接自身的 127.0.0.1	改为同源或环境地址，验证 WS/WSS
P0	公网 HTTPS 未确认	登录和业务数据可能明文传输	配置域名、证书、HTTPS 和 WSS

优先级	风险项	风险说明	应对策略
P0	内存 Token	重启失效、无过期、多实例不共享	引入规范会话或 Token 机制
P1	订单状态不完整	当前创建后直接完成，与真实流程不符	实现状态机、事务、幂等和权限检查
P1	购物车与地址回归结果未归档	服务端化后存在归属和迁移回归风险	执行接口与 E2E 用例并保存流水线报告
P1	数据库迁移分散	Mapper 运行期 DDL 可能导致环境漂移	引入 Flyway/Liquibase
P1	治理与售后不完整	举报、售后、仲裁不能形成闭环	先补需求与数据模型，再实现测试
P1	缺少完整监控	性能和故障难以定位	增加指标、结构化日志和告警
P2	AI 外部依赖	超时、限流、费用和可用性波动	限流、超时、降级和费用告警
P2	单机部署单点	主机故障造成整体不可用	先保证备份恢复，再按 SLA 升级

10.2 未决事项

- 1. 公网目标主机、域名、DNS、HTTPS 证书和安全组的最终方案；
- 2. 登录态采用服务端会话还是标准化 JWT，以及过期与撤销策略；
- 3. 订单状态机、支付模拟、确认收货和售后是否纳入本次最终验收；
- 4. 举报、申诉、仲裁和通用审计的交付范围；
- 5. 是否根据压力测试结果引入 Redis；
- 6. 是否接入对象存储和正式 AI 服务；
- 7. 何时将数据库兼容 DDL 迁移到专用版本迁移工具；
- 8. 监控、日志、指标和链路追踪采用本地组件还是云平台服务。

11 需求与设计对应关系

需求域	对应设计模块	当前实现说明
用户与信用管理能力	M1 用户与权限模块	注册登录、资料、实名、服务端地址和信用主要能力已实现；登录态仍需生产化
商品与内容管理能力	M2 商品与内容模块、M8 基础支撑模块	商品、店铺、媒体和商品审核主要能力已实现
搜索与推荐能力	M3 搜索与推荐模块、M7 AI 扩展模块	基础筛选排序和 AI 辅助已实现；高级推荐待验证
交易与评价能力	M4 购物车与订单模块、M5 售后与评价模块	服务端购物车/地址接口已实现但当前结算页未接入；基础订单和评价已实现，完整状态机与售后未实现

需求域	对应设计模块	当前实现说明
社区互动能力	商品/内容域中的社区模块	话题、帖子、评论、点赞与动作数据已实现
平台治理与审核能力	M6 平台治理模块、M8 基础支撑模块	用户、实名和商品审核部分实现；举报、仲裁和完整审计待实现
消息与沟通能力	M8 基础支撑模块中的消息子系统	会话、历史消息和 WebSocket 已实现；远程地址需修复
AI 交互能力	M7 AI 扩展模块	议价、发布建议和助手接口已实现，依赖外部服务
部署与质量能力	M8 基础支撑模块	Docker Compose、Kubernetes、Nginx 和分层自动化测试资产已具备；实际目标环境结果待验收